

YOLOv8 Improved with DCNv3

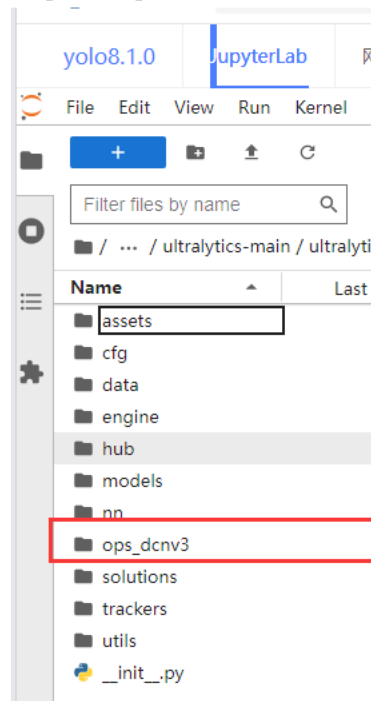
This paper details how to integrate the DCNv3 module into YOLOv8, including downloading YOLOv8 and InternImage from official repositories, copying the ops_dcnv3 folder to the project directory, modifying YOLOv8 source code to add DCNv3 related modules, initializing configurations and running on CUDA. Following these steps, DCNv3 can be successfully applied to YOLOv8 training. It delivers obvious mAP improvement! Fully tested and functional.

1 Install YOLOv8

Due to the Ultralytics system updating to YOLOv26, the project team has uploaded the YOLOv8 model. Please download the file named ultral-main from this project, which contains the YOLOv8 model.

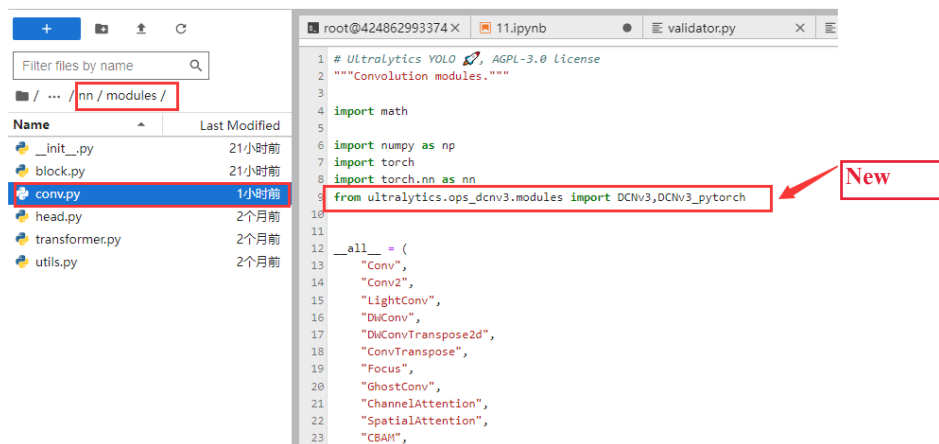
2 Copy & Compile ops_dcnv3 Module

Copy folder [ops_dcnv3](#) into ultralytics /root folder, Switch terminal path to [ultralytics-main/ultralytics/ops_dcnv3](#), Run compile script: sh make.sh

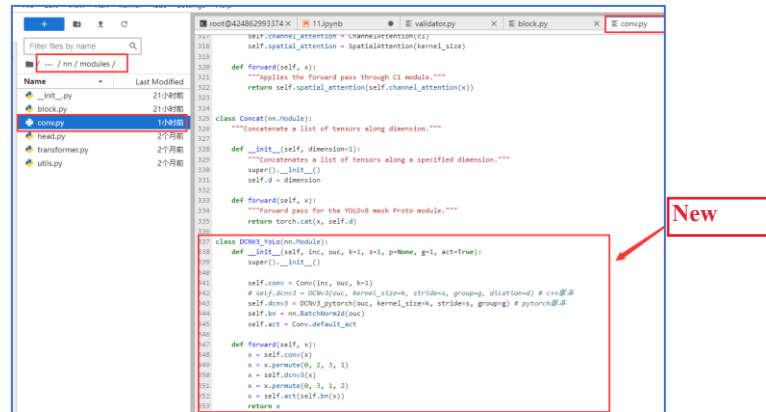


3 Add DCNv3_YoLo Module in nn/modules/conv.py

Step 1 Import Dependencies at file head



Step 2 Add DCNv3_YoLo Class at file tail



The code is as follows

class DConv3_Yolo(nn.Module):

def __init__(self, inc, ouc, k=1, s=1, p=None, g=1, act=True):

super().__init__()

self.conv = Conv(inc, ouc, k=1)

self.dcnv3 = DConv3(ouc, kernel_size=k, stride=s, group=g, dilation=d) # C++ CUDA accelerated version

self.dcnv3 = DConv3_pytorch(ouc, kernel_size=k, stride=s, group=g) # Pure PyTorch version

self.bn = nn.BatchNorm2d(ouc)

self.act = nn.SiLU() if act is True else (act if isinstance(act, nn.Module) else nn.Identity())

def forward(self, x):

x = self.conv(x)

x = x.permute(0, 2, 3, 1)

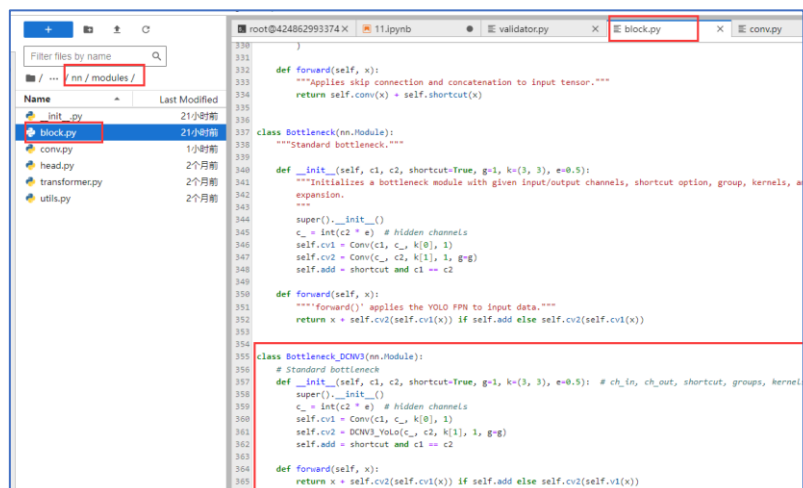
x = self.dcnv3(x)

x = x.permute(0, 3, 1, 2)

x = self.act(self.bn(x))

return x

4 Add Bottleneck_DConv3 in nn/modules/block.py

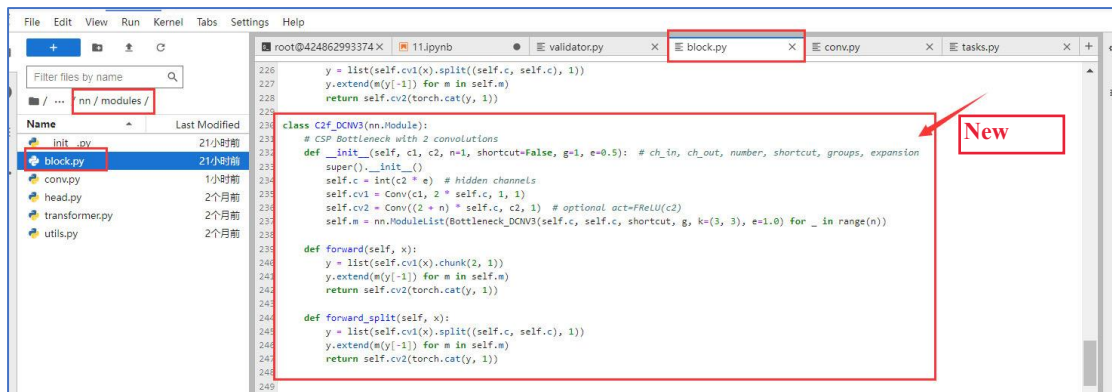


The code is as follows

```
class Bottleneck_DCNV3(nn.Module):
    # Standard Bottleneck replaced second conv with DCNV3_YoLo
    def __init__(self, c1, c2, shortcut=True, g=1, k=(3,3), e=0.5):
        super().__init__()
        c_ = int(c2 * e) # hidden channel count
        self.cv1 = Conv(c1, c_, k[0], 1)
        self.cv2 = DCNV3_YoLo(c_, c2, k[1], 1, g=g)
        self.add = shortcut and c1 == c2
```

```
def forward(self, x):
    return x + self.cv2(self.cv1(x)) if self.add else self.cv2(self.cv1(x))
```

5 Add C2f_DCNV3 in nn/modules/block.py



The code is as follows

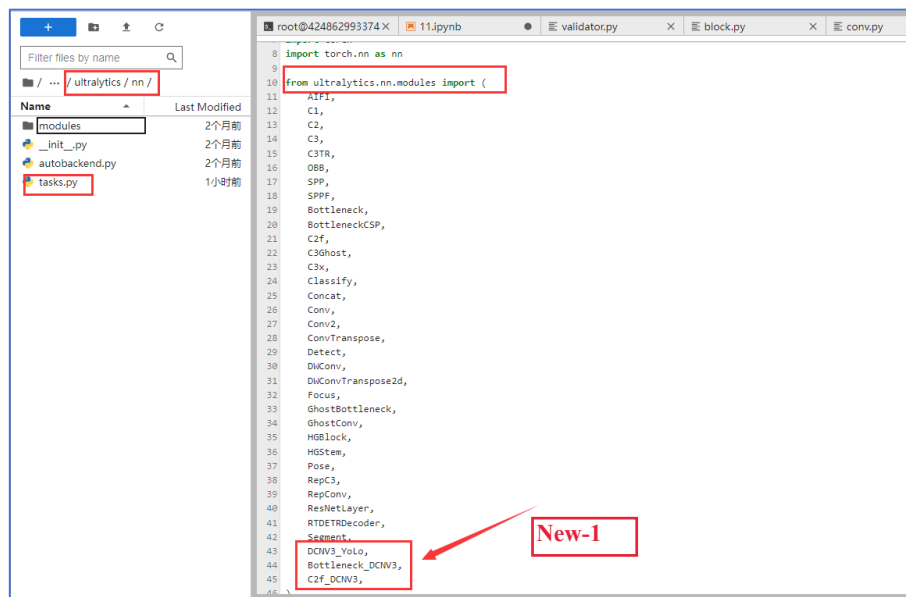
```
class C2f_DCNV3(nn.Module):
    # Modified C2f with DCNV3 Bottleneck stack
    def __init__(self, c1, c2, n=1, shortcut=False, g=1, e=0.5):
        super().__init__()
        self.c = int(c2 * e) # hidden channels
        self.cv1 = Conv(c1, 2 * self.c, 1, 1)
        self.cv2 = Conv((2 + n) * self.c, c2, 1)
        self.m = nn.ModuleList(Bottleneck_DCNV3(self.c, self.c, shortcut, g, k=(3, 3),
        e=1.0) for _ in range(n))
    def forward(self, x):
        y = list(self.cv1(x).chunk(2, 1))
        y.extend(m(y[-1]) for m in self.m)
        return self.cv2(torch.cat(y, 1))
    def forward_split(self, x):
        y = list(self.cv1(x).split((self.c, self.c), 1))
        y.extend(m(y[-1]) for m in self.m)
        return self.cv2(torch.cat(y, 1))
```

6 Register Custom Modules in nn/tasks.py

6.1 Import modules at file header

Add DCNV3_YoLo, Bottleneck_DCNV3, C2f_DCNV3 into the import tuple from

ultralytics.nn.modules



6.2 Module type recognition in parse_model() line 784

Add three custom modules into the model judge tuple, enable yaml config parser to recognize new layers.



6.3 Parameter adaptation logic line 793

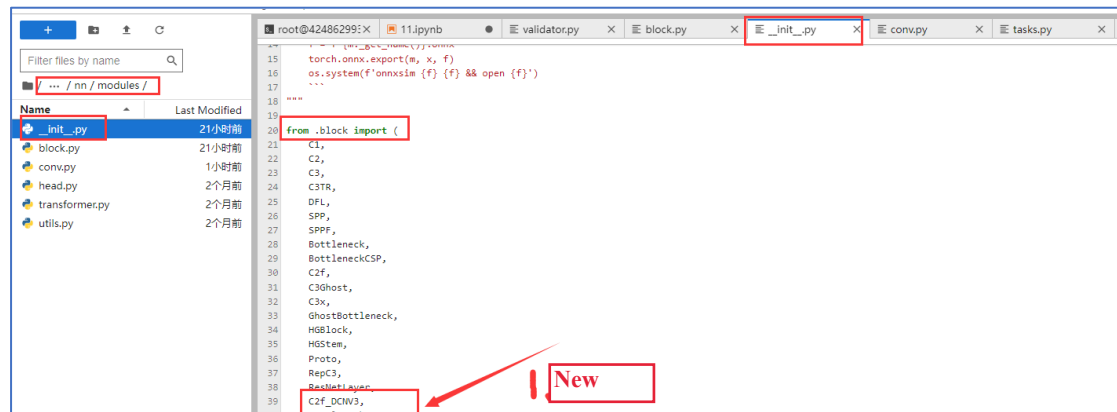
Custom modules share same parameter rule as C2f/C3 series, auto set repeat count n=1 when parsing yaml.



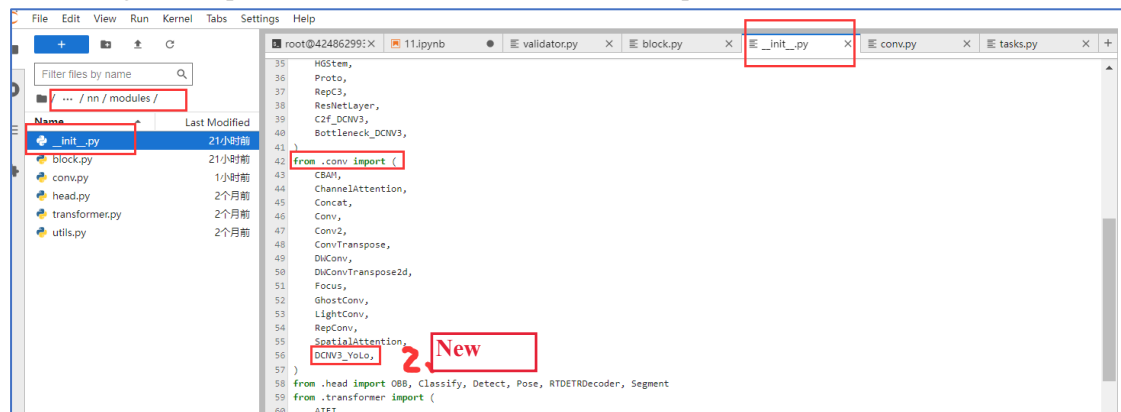
7 Global Export Initialization

7.1 Append DCNV3_YoLo to __all__ list in block.py

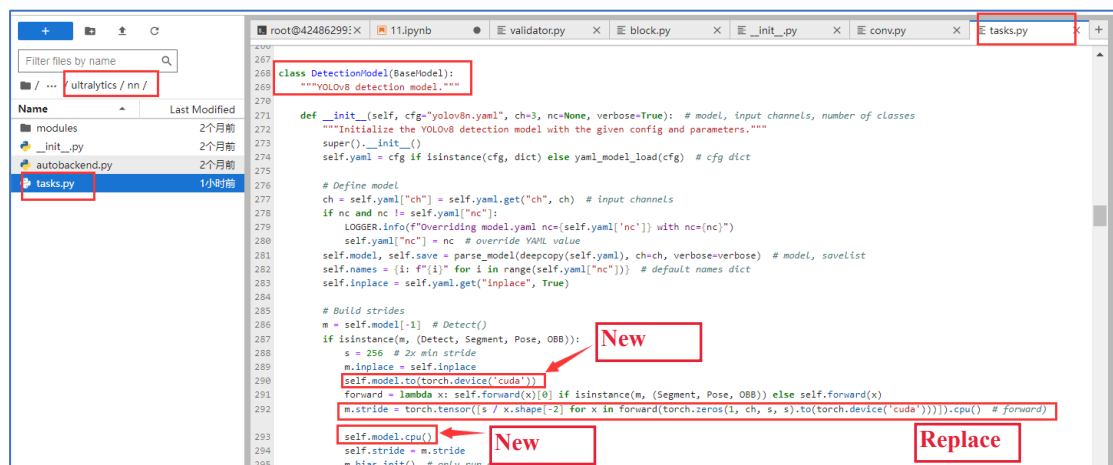
Import & export all 3 modules in ultralytics/nn/modules/init.py



7.2 Allow global import of custom modules without full file path.



8 Adapt CUDA runtime in nn/tasks.py



The code is as follows

```
# model /yolo.py DetectionModel class
self.model.to(torch.device('cuda'))
m.stride = torch.tensor([s / x.shape[-2] for x in forward(torch.zeros(1, ch, s,
s).to(torch.device('cuda')))]).cpu()
self.model.cpu()
```

Then follow the YOLOv8 training process to start training the model, and the training of the above model can be completed using the dataset, obtaining the best. file which can be used for engineering applications.